

The Standard of Data Structure – Basic

자료구조의 정석 기초

02 배열

목차 02 배열

1. 배열의 특징

2. 배열 메소드

1. 배열의 특징



- 배열이란?

- 연속된 메모리 공간에 순차적으로 데이터를 저장하는 선형 자료구조

- Python 등을 제외하면 대부분의 프로그래밍 언어에서 동일 타입의 데이터를 저장

- int 타입인 경우 정수 요소만 저장

```
// 배열 크기 정의
int size = 5;

// 배열 선언
int arr[size];
```

C++

```
# 배열 크기 정의
size = 5

# 배열 선언
arr1 = [0] * size

# Python의 경우 다양한 자료형의 데이터를 한 번에 저장 가능
arr2 = [1, 'str', 23.3]
```

Python

```
// 배열 크기 정의
int size = 5;

// 배열 선언
int[] arr = new int[size];
```

Java



1. 연속된 메모리 공간에 데이터들이 순차적으로 저장됨

- 실제 메모리 상에서 물리적으로 데이터가 연속적으로 저장
- → indexing 및 slicing이 가능

2. 인덱스는 0부터 시작

- 1부터 시작하지 않음에 주의!
- 마지막 인덱스는 배열 요소의 개수 - 1
- 각 요소는 인덱스를 통해 접근 가능
 - 5번째 요소를 접근하고 싶으면 인덱스 4를 사용

	0	1	2	3	4	5	6	7
myList	3	4	2	5	1	2	6	3

배열에 저장되는 값들은 순서를 나타내는 번호를 가짐
→ 인덱스



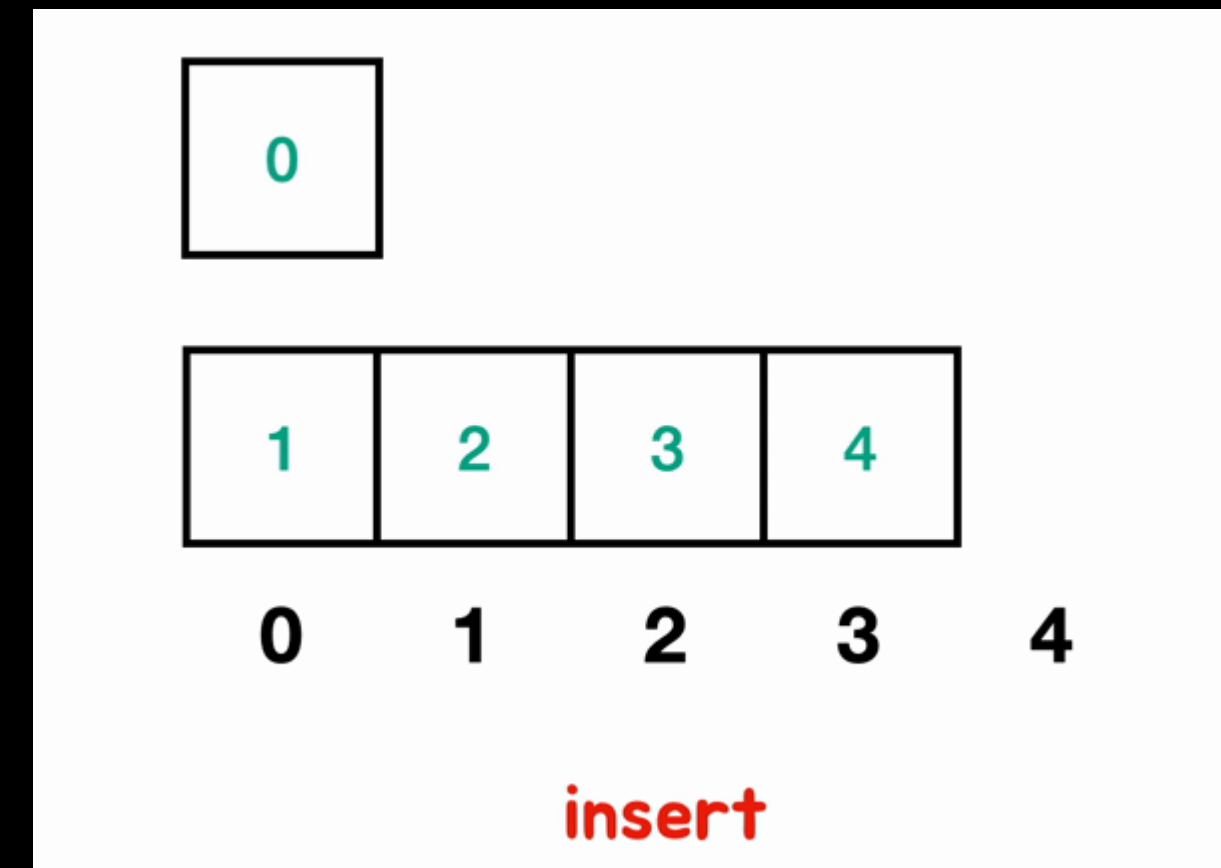
1. 인덱스를 이용한 접근이 가능하기 때문에 **검색이 빠름**
2. 연속적이므로 **메모리 공간을 효율적으로 사용** 가능
 - 요소들이 순차적으로 저장되므로 캐시 지역성이 좋음
3. 간단하고 사용 및 구현이 쉬움
4. 참조를 위한 추가적인 메모리가 필요하지 않음



1. 배열을 선언한 후에는 할당된 **정적 메모리** 때문에 **크기를 변경할 수 없음**

2. 삽입, 삭제에 **비용이 많이 듦**

- 항상 메모리가 순차적으로 이어져 있어야 하기 때문에,
- 다른 모든 요소를 이동해줘야 함



3. 오버플로우나 **저장 공간의 낭비**를 초래할 수 있음

- 삽입과 삭제가 동적으로 발생하는 상황에서 적절한 배열의 크기를 미리 결정하는 것이 어려움



- 순차적인 데이터를 저장하며, 순서가 중요한 경우
- 특정 요소를 빠르게 읽어야 할 때
- 데이터의 사이즈가 자주 바뀌지 않으며, 요소의 추가 및 삭제가 적게 발생할 때
- 다차원의 데이터를 다룰 때



```
#include <iostream>

int main() {
    // 2차원 배열 선언
    int arr2D[3][3]; // 3x3 2차원 배열 선언

    // 3차원 배열 선언
    int arr3D[3][3][3]; // 3x3x3 3차원 배열 선언

    return 0;
}
```

C++

```
# 2차원 배열 선언
arr2D = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]

# 3차원 배열 선언
arr3D = [[[0, 0, 0], [0, 0, 0], [0, 0, 0]],
          [[0, 0, 0], [0, 0, 0], [0, 0, 0]],
          [[0, 0, 0], [0, 0, 0], [0, 0, 0]]]
```

Python

```
public class Main {
    public static void main(String[] args) {
        // 2차원 배열 선언
        int[][] arr2D = new int[3][3]; // 3x3 2차원 배열 선언

        // 3차원 배열 선언
        int[][][] arr3D = new int[3][3][3]; // 3x3x3 3차원 배열 선언
    }
}
```

Java

2. 배열 메소드



- 원하는 인덱스에 접근하는 시간복잡도는 $O(1)$
- 데이터 삽입에 걸리는 시간복잡도는 $O(n)$
 - 삽입을 위해 기존 데이터를 이동시켜줘야 하기 때문
- 데이터 삭제에 걸리는 시간복잡도는 $O(n)$
 - 마찬가지로 삭제 후, 기존 데이터를 이동시켜줘야 하기 때문



• 배열 선언 예시

```
int numbers[5]; // 크기가 5인 정수형 배열 선언
```

C++

```
numbers = [1, 2, 3, 4, 5] # 리스트를 사용하여 배열 선언
```

Python

```
int[] numbers = new int[5]; // 크기가 5인 정수형 배열 선언
```

Java

• 배열에 값 할당

- 인덱스를 사용해 특정 위치에 값 대입

```
arr[1] = 2;  
arr[3] = 5;  
arr[4] = 7;
```




• 배열의 길이

```
#include <iostream>

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int length = sizeof(arr) / sizeof(arr[0]);
    std::cout << "Array length: " << length << std::endl;
    return 0;
}
```

C++

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        int length = arr.length;
        System.out.println("Array length: " + length);
    }
}
```

Java

```
arr = [1, 2, 3, 4, 5]
length = len(arr)
print("Array length:", length)
```

Python



- 배열 반복문

- 배열의 모든 요소에 순차적으로 접근하기 위해 반복문을 사용

```
for (int i = 0; i < length; i++) {  
    std::cout << numbers[i] << std::endl;  
}
```

C++

```
for (int i = 0; i < numbers.length; i++) {  
    System.out.println(numbers[i]);  
}
```

Java

```
numbers = [1, 2, 3, 4, 5]  
for num in numbers:  
    print(num)
```

Python



- C++에서는 vector, Java는 ArrayList
- Python에서 list는 이미 동적 배열

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> numbers; // 동적 배열 선언

    // 데이터 삽입
    numbers.push_back(1);
    numbers.push_back(2);
    numbers.push_back(3);

    // 데이터 삭제
    numbers.pop_back(); // 마지막 요소 삭제

    return 0;
}
```

C++

```
numbers = [1, 2, 3] # 리스트 선언

# 데이터 삽입
numbers.append(4) # 리스트 끝에 요소 추가

# 데이터 삭제
numbers.pop() # 리스트 마지막 요소 삭제
```

Python

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<Integer>(); // ArrayList 선언

        // 데이터 삽입
        numbers.add(1);
        numbers.add(2);
        numbers.add(3);

        // 데이터 삭제
        numbers.remove(numbers.size() - 1); // 마지막 요소 삭제
    }
}
```

Java